

ACCRA TECHNICAL UNIVERSITY

Department of Computer Science

BCS 404: Introduction to Data Science with Python

PROJECT WORK

Submitted by:

Youngson Ahiase

01256644B

Table of Contents

1. Introduction.....	3
2. Dataset Description.....	3
2.1 Dimensions.....	3
2.2 Variables	3
3. Methodology.....	4
3.1 Data Acquisition and Cleaning.....	4
3.2 Data Visualisation.....	4
3.3 Statistical Analysis.....	5
3.4 Machine Learning.....	5
4. Results.....	5
4.1 Descriptive Statistics.....	5
4.2 Frequency Distributions	6
4.3 Data Visualisation.....	6
4.4 Correlation Analysis	9
4.5 Machine Learning Results.....	10
5. Discussion	10
5.1 Major Findings	10
5.2 Statistical Insights.....	10
5.3 Machine Learning Results.....	11
5.4 Limitations of the Study	11
5.5 Recommendations.....	11
6. Conclusion	11
7. References.....	12
8. Appendix: Python Code	12

1. Introduction

The sinking of the RMS Titanic on 15 April 1912 remains one of the most widely studied maritime disasters in history, not only for its historical significance but also for the richness of the passenger data that survives it. This project uses the Titanic passenger dataset, made publicly available through the Kaggle "Titanic: Machine Learning from Disaster" competition, to demonstrate practical competence in data analysis and machine learning using the Python programming language.

The purpose of this project is to apply the core Python data-science stack — Pandas, NumPy, Matplotlib, Seaborn, and Scikit-Learn — to a real-world dataset in order to (i) acquire and inspect raw data, (ii) clean and prepare it for analysis, (iii) visualise and statistically summarise the relationships between passenger attributes and survival, and (iv) build a supervised machine learning model capable of predicting whether a passenger survived the disaster. The project therefore spans the full, practical data-science workflow: from raw CSV file to a trained and evaluated classification model.

This report documents the methodology, results, and interpretation of each stage of the project. Section 2 describes the dataset. Section 3 outlines the methodology used for cleaning, visualisation, statistical analysis and modelling. Section 4 presents the results, Section 5 discusses their implications, and Section 6 concludes the report with a summary, limitations, and recommendations for future work. The complete Python source code used to generate all outputs presented in this report is provided in the Appendix.

2. Dataset Description

The dataset used in this project is the Titanic training dataset (train.csv), obtained from the Kaggle competition page at <https://www.kaggle.com/competitions/titanic/data>. It contains demographic and travel information for a subset of the passengers aboard the Titanic, together with a binary label indicating whether each passenger survived.

2.1 Dimensions

The dataset consists of 891 observations (rows) and 12 variables (columns).

2.2 Variables

The table below summarises the variables present in the raw dataset.

Variable	Description	Type
PassengerId	Unique identifier for each passenger	Integer
Survived	Survival indicator (0 = No, 1 = Yes) — target variable	Integer
Pclass	Ticket / passenger class (1 = 1st, 2 = 2nd, 3 = 3rd)	Integer
Name	Full name of the passenger	Text
Sex	Gender of the passenger	Categorical

Variable	Description	Type
Age	Age in years	Float
SibSp	Number of siblings / spouses aboard	Integer
Parch	Number of parents / children aboard	Integer
Ticket	Ticket number	Text
Fare	Passenger fare paid	Float
Cabin	Cabin number	Text
Embarked	Port of embarkation (C, Q, S)	Categorical

Of the 12 variables, Age (714 non-null of 891) and Cabin (204 non-null of 891) contain substantial missing values, and Embarked has 2 missing values. These gaps are addressed in the data-cleaning stage described in Section 3.1.

3. Methodology

The project followed a standard applied data-science pipeline implemented in a Jupyter Notebook using Python 3, with Pandas and NumPy for data manipulation, Matplotlib and Seaborn for visualisation, and Scikit-Learn for machine learning.

3.1 Data Acquisition and Cleaning

The dataset was loaded directly into a Pandas DataFrame from the CSV file. An initial inspection used `.shape`, `.columns`, `.head()` and `.info()` to confirm the dimensions, variable names, sample records, and data types described in Section 2.

Missing-value detection was performed using `df.isnull().sum()`. Two variables required treatment:

- Age (177 missing values): imputed using the median age of the dataset. The median was preferred over the mean because the age distribution is right-skewed and the median is more robust to outliers.
- Embarked (2 missing values): imputed using the mode (the most frequently occurring port), an appropriate strategy for a categorical variable with very few missing entries.

The Cabin variable was retained in the raw dataset but excluded from later modelling because more than 77% of its values are missing, making reliable imputation impractical. Duplicate-row detection was carried out using `df.duplicated().sum()`; no duplicate observations were found, so no rows were removed.

3.2 Data Visualisation

Six visualisations were produced using Matplotlib and Seaborn to explore the distribution of, and relationships between, key variables: a histogram of passenger ages, a bar chart of passenger class

distribution, a boxplot of age by passenger class, a scatter plot of age versus fare (coloured by survival), a correlation heatmap of numerical features, and a pairplot of selected numerical variables. Each chart was produced with a descriptive title and labelled axes, and is presented with interpretation in Section 4.

3.3 Statistical Analysis

Descriptive statistics (count, mean, standard deviation, quartiles, minimum and maximum) were computed for all numerical variables using `df.describe()`. Frequency distributions were computed for the categorical variables `Pclass` and `Embarked` using `value_counts()`. A Pearson correlation matrix was computed across all numerical variables using `df.corr()`, from which the strongest positive and strongest negative pairwise correlations were identified programmatically.

3.4 Machine Learning

A binary classification model was built to predict the `Survived` variable. `Pclass`, `Age`, `SibSp`, `Parch`, `Fare`, `Sex` and `Embarked` were selected as predictors on the basis of their plausible relationship to survival (socio-economic status, family structure, and demographic factors were all recorded as relevant in historical accounts of the disaster). The categorical variables `Sex` and `Embarked` were converted to numerical form using one-hot encoding (`pd.get_dummies`, with `drop_first=True` to avoid the dummy-variable trap).

The resulting feature matrix was split into training (80%) and testing (20%) subsets using `train_test_split` with a fixed `random_state` of 42 for reproducibility, giving 712 training and 179 testing observations. Any residual missing values in the `Age` feature after the split were imputed with the training-set median using Scikit-Learn's `SimpleImputer`, fitted on the training data only and applied to both sets to avoid data leakage.

A Logistic Regression classifier (Scikit-Learn's `LogisticRegression`, `liblinear` solver, `max_iter = 200`) was trained on the training set and used to predict survival on the held-out test set. Model performance was evaluated using accuracy, a confusion matrix, and a full classification report (precision, recall, and F1-score per class).

4. Results

4.1 Descriptive Statistics

Table 2 summarises the descriptive statistics for the key numerical variables after cleaning.

Statistic	Age	Fare	SibSp	Parch
Count	891	891	891	891
Mean	29.36*	32.20	0.52	0.38
Std. Dev.	13.02*	49.69	1.10	0.81
Min	0.42	0.00	0	0

Statistic	Age	Fare	SibSp	Parch
25%	22.00*	7.91	0	0
50% (Median)	28.00	14.45	0	0
75%	35.00*	31.00	1	0
Max	80.00	512.33	8	6

On average, passengers paid a fare of about 32.20 (in the currency units of the original dataset), though the very large standard deviation (49.69) and maximum (512.33) relative to the median (14.45) indicate a strongly right-skewed fare distribution driven by a small number of very expensive first-class tickets.

4.2 Frequency Distributions

Passenger class (Pclass) was distributed as follows: 3rd class — 491 passengers (55.1%), 1st class — 216 passengers (24.2%), 2nd class — 184 passengers (20.6%). The port of embarkation (Embarked) was distributed as: Southampton (S) — 644 passengers (72.3%), Cherbourg (C) — 168 passengers (18.9%), Queenstown (Q) — 77 passengers (8.6%). Southampton was clearly the dominant embarkation point, and more than half of all passengers travelled in 3rd class.

4.3 Data Visualisation

4.3.1 Histogram of Passenger Ages

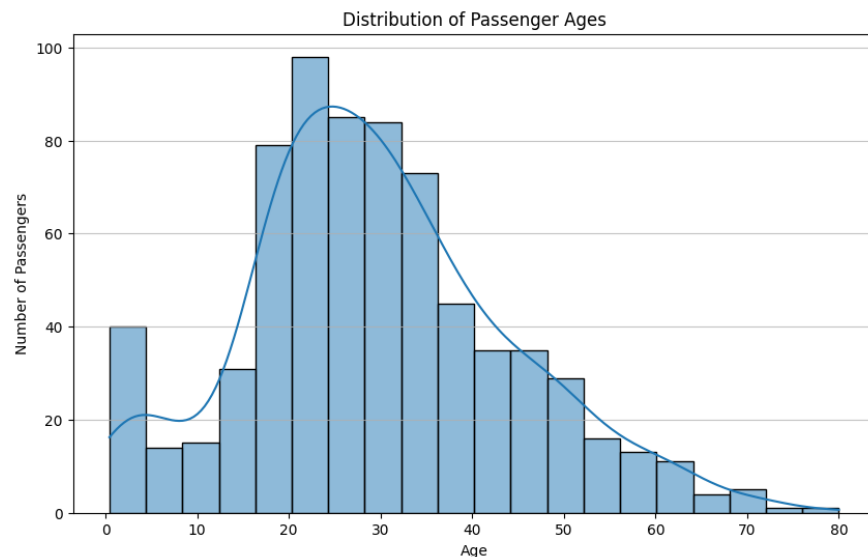


Figure 1. Distribution of passenger ages.

The age distribution is unimodal and right-skewed, with a pronounced concentration of passengers in their 20s and 30s. A visible secondary group of infants and young children is also present, while comparatively few passengers were aged above 60.

4.3.2 Passenger Class Distribution

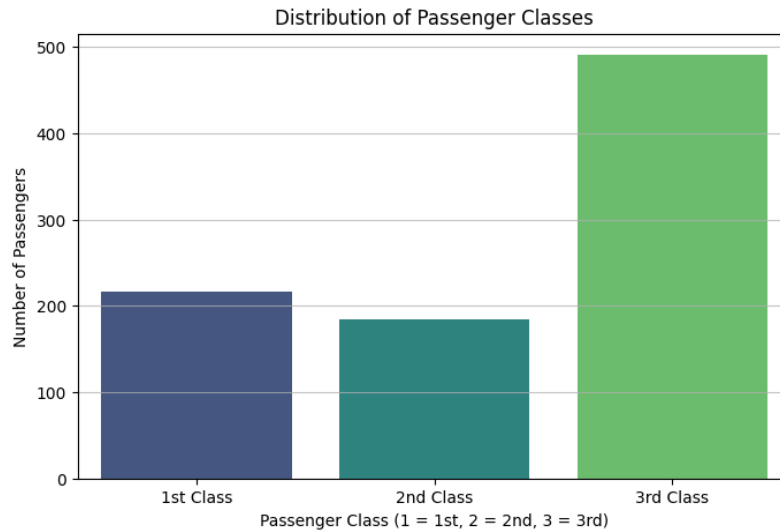


Figure 2. Distribution of passengers by class.

Third-class passengers substantially outnumber those in first and second class, consistent with the fact that third class represented the most affordable accommodation on board and therefore attracted the largest number of travellers.

4.3.3 Boxplot of Age by Passenger Class

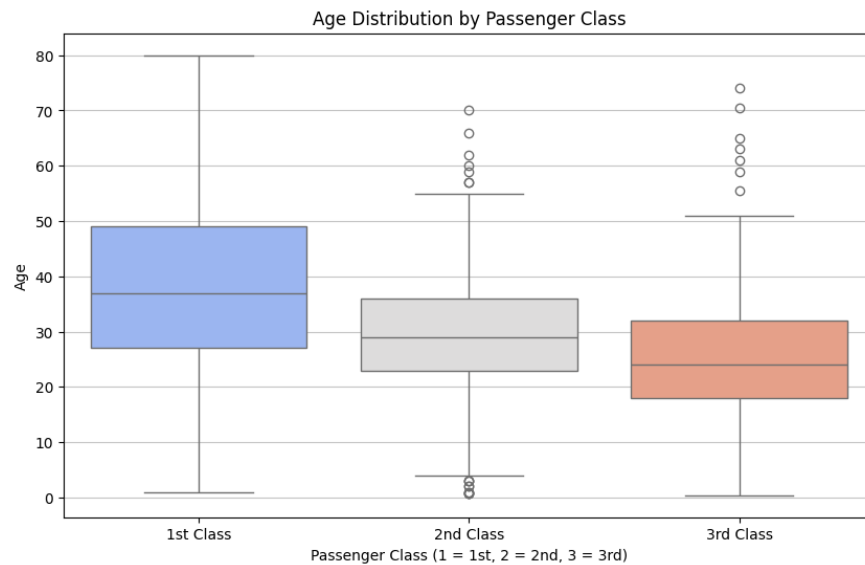


Figure 3. Age distribution across passenger classes.

Median passenger age decreases monotonically from first to third class. First-class passengers were, on average, older and likely more affluent, while third-class passengers were predominantly younger adults and families seeking passage at lower cost.

4.3.4 Scatter Plot of Age versus Fare

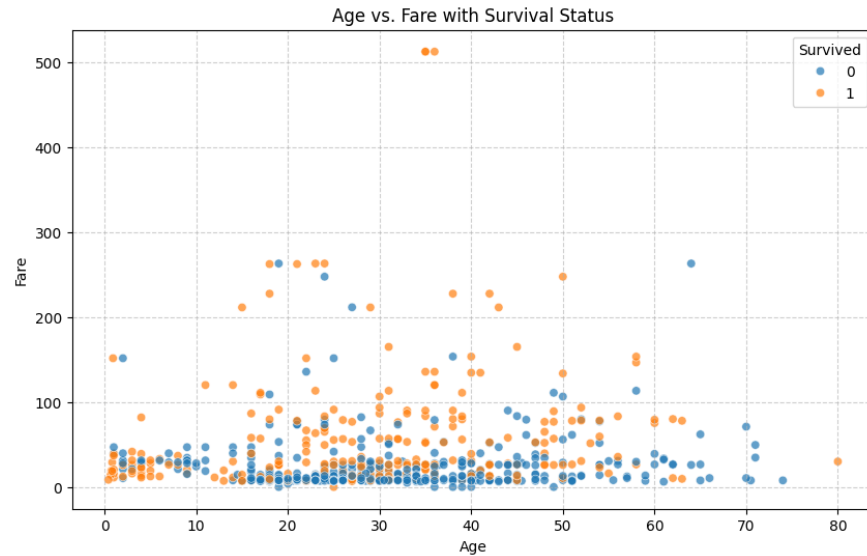


Figure 4. Age versus fare, coloured by survival status.

No strong linear relationship is apparent between age and fare. However, a cluster of high-fare passengers — largely concentrated at fares above roughly 100 — shows a visibly higher proportion of survivors, while the dense cluster of low-fare passengers across all ages shows a higher proportion of non-survivors.

4.3.5 Correlation Heatmap

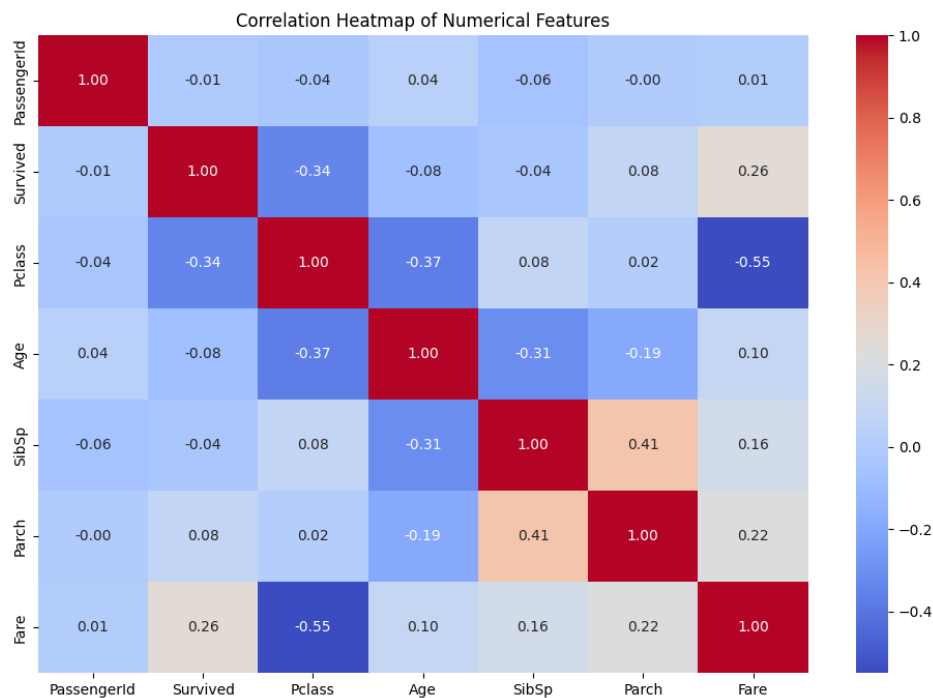


Figure 5. Correlation heatmap of numerical features.

The heatmap confirms a moderate negative correlation between Pclass and Fare (higher-numbered, lower-status classes paid less), a moderate positive correlation between Fare and Survived, and a moderate negative correlation between Pclass and Survived. SibSp and Parch are moderately positively correlated with one another, reflecting shared family-size effects.

4.3.6 Pairplot of Selected Numerical Variables

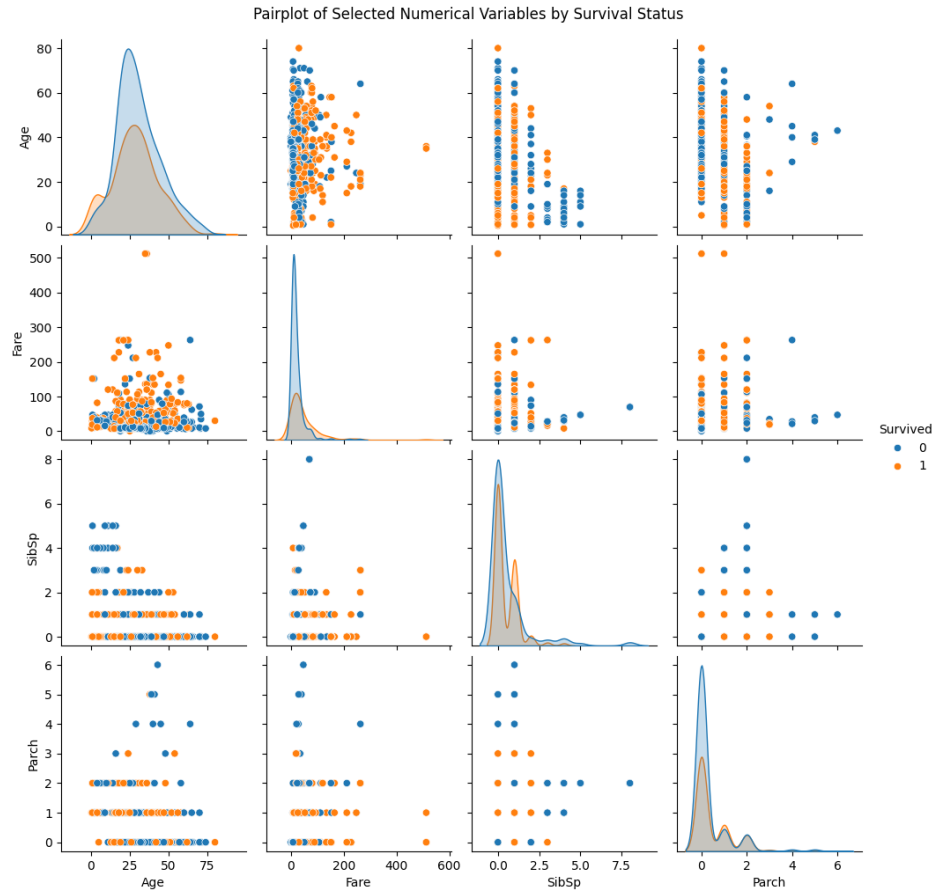


Figure 6. Pairplot of Age, Fare, SibSp, Parch and Survived.

The pairplot reinforces the univariate and bivariate patterns observed above: survivors and non-survivors show broadly overlapping age distributions but a visibly different fare distribution, with survivors skewed toward higher fares.

4.4 Correlation Analysis

The strongest positive pairwise correlation in the numerical feature set was between SibSp and Parch ($r = 0.41$), reflecting that passengers travelling with siblings or a spouse were also more likely to be travelling with parents or children, i.e. as part of a larger family unit. The strongest negative pairwise correlation was between Pclass and Fare ($r = -0.55$), confirming that passenger class and ticket price are closely linked — lower class numbers (higher status, e.g. 1st class) are associated with higher fares.

With specific respect to the target variable, Survived correlated negatively with Pclass ($r = -0.34$) and positively with Fare ($r = 0.26$), indicating that both socio-economic proxies were meaningfully — though not overwhelmingly — associated with survival.

4.5 Machine Learning Results

The Logistic Regression classifier achieved an overall accuracy of 78% on the held-out test set (179 passengers). The confusion matrix and classification report are shown below.

	Predicted: No Survival	Predicted: Survival
Actual: No Survival	89	16
Actual: Survival	23	51

Table 3. Confusion matrix for the Logistic Regression model.

Class	Precision	Recall	F1-score	Support
0 (Did not survive)	0.79	0.85	0.82	105
1 (Survived)	0.76	0.69	0.72	74
Accuracy			0.78	179
Macro avg	0.78	0.77	0.77	179
Weighted avg	0.78	0.78	0.78	179

Table 4. Classification report for the Logistic Regression model.

The model is noticeably better at correctly identifying non-survivors (recall = 0.85) than survivors (recall = 0.69); of the 74 passengers who actually survived in the test set, 23 (31%) were misclassified as non-survivors (false negatives). This asymmetry is discussed further in Section 5.

5. Discussion

5.1 Major Findings

The exploratory analysis confirms several patterns consistent with well-known historical accounts of the Titanic disaster. Passengers were predominantly young adults, third class was the largest travel group, and fares were highly right-skewed, driven by a small number of expensive first-class tickets. Passenger class emerged as a strong structural variable, correlating with both fare (strongly negative) and age (moderately negative), and — critically — with survival.

5.2 Statistical Insights

The correlation analysis provides quantitative support for the qualitative narrative that "women and children first" and socio-economic status shaped survival outcomes. Pclass and Fare are both linked to Survived in directions consistent with the hypothesis that wealthier, higher-class passengers had preferential access to lifeboats. The moderate (rather than strong) magnitude of these correlations ($|r| \approx 0.26\text{--}0.34$) indicates that class and fare were contributing factors rather than sole determinants of survival

— other unmeasured or weakly-measured factors (such as Sex, which is categorical and was not included in the numerical correlation matrix, but was used in the ML model) also played an important role.

5.3 Machine Learning Results

The Logistic Regression model's 78% accuracy represents a reasonable baseline for this classification problem, comfortably outperforming a naive majority-class baseline (which would achieve approximately 61.6% by always predicting "did not survive"). The precision and recall figures show that the model is more conservative in predicting survival than non-survival: it favours the majority class somewhat, which is reflected in a lower recall (0.69) for the "survived" class. In a context where correctly identifying survivors is important (for example, historical or humanitarian analysis), this false-negative rate would be worth addressing through class-balancing techniques or alternative classification thresholds.

5.4 Limitations of the Study

- The Cabin variable, which could plausibly have improved prediction (via deck-level proximity to lifeboats), was excluded due to a very high proportion (>77%) of missing values.
- Median imputation for Age, while a common and defensible baseline strategy, does not capture the underlying uncertainty in the estimate and may understate variance; a more sophisticated approach (e.g. regression-based or group-median imputation by Pclass/Sex) could improve accuracy.
- A single linear model (Logistic Regression) was evaluated; no comparison against non-linear models (e.g. Random Forest, Gradient Boosting) or hyperparameter tuning was performed, so it is unclear whether 78% represents close to the practical ceiling for this feature set.
- The dataset itself is a historical sample of 891 passengers out of the roughly 2,224 people aboard, and may not be perfectly representative of the full passenger and crew population.

5.5 Recommendations

- Engineer additional features such as family size (SibSp + Parch), title extracted from the Name field (e.g. Mr, Mrs, Master), and a binary "has cabin data" indicator.
- Compare Logistic Regression against tree-based ensemble methods (Random Forest, XGBoost) and evaluate with cross-validation rather than a single train/test split.
- Apply class-balancing techniques (e.g. `class_weight='balanced'`, SMOTE) to reduce the false-negative rate for the survived class.
- Conduct formal hypothesis tests (e.g. chi-square test of independence between Sex/Pclass and Survived) to complement the correlation-based analysis with statistical significance testing.

6. Conclusion

This project applied a complete, practical data-science workflow — acquisition, cleaning, visualisation, statistical analysis, and machine learning — to the Titanic passenger dataset using Python. The exploratory and statistical analyses confirmed that passenger class and fare, both proxies for socio-economic status, were meaningfully associated with survival, alongside demographic patterns in age and family

composition. A Logistic Regression classifier trained on these and related features achieved 78% accuracy on unseen test data, correctly identifying the majority of both survivors and non-survivors, while showing room for improvement in recall for the survivor class. The exercise demonstrates that even a relatively simple, interpretable model can extract meaningful predictive signal from historical demographic data, while also illustrating the importance of careful data cleaning, feature selection, and evaluation in building a trustworthy classification pipeline. Future work along the lines recommended in Section 5.5 — richer feature engineering, model comparison, and rigorous statistical testing — would likely yield further improvements in predictive performance and analytical depth.

7. References

Kaggle. (n.d.). Titanic – Machine Learning from Disaster. Kaggle Competitions. Retrieved July 2026, from <https://www.kaggle.com/competitions/titanic/data>

McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 51–56.

Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.

Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90–95.

Waskom, M. L. (2021). seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60), 3021.

Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

8. Appendix: Python Code

The complete Python source code used to acquire, clean, visualise, statistically analyse, and model the Titanic dataset (as executed in the accompanying Jupyter Notebook) is reproduced below in full.

Task 1: Data Acquisition

```
import pandas as pd
df = pd.read_csv('train.csv')
print("Dataset loaded successfully.")
```

1. Dataset dimensions

```
print(f"Dataset dimensions (rows, columns): {df.shape}")
```

2. Column names

```
print("Column names:")
for col in df.columns:
    print(f"- {col}")
```

3. First five observations

```
display(df.head())
```

4. Data types

```
print("Data types:")
display(df.info())
```

Task 2: Data Cleaning**### Detect missing values**

```
print("Missing values before cleaning:")
print(df.isnull().sum())
```

Handle missing values appropriately**Impute 'Age' with the median**

```
df['Age'].fillna(df['Age'].median(), inplace=True)
```

Impute 'Embarked' with the mode

```
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
```

```
print("Missing values after cleaning:")
print(df.isnull().sum())
```

```
print("\nDataFrame info after cleaning:")
df.info()
```

Detect duplicated observations

```
duplicated_rows = df.duplicated().sum()
print(f"Number of duplicated rows before removal: {duplicated_rows}")
```

Remove duplicates if any exist

```
if duplicated_rows > 0:
    df.drop_duplicates(inplace=True)
    print(f"Number of duplicated rows after removal: {df.duplicated().sum()}")
else:
```

```
print("No duplicated rows found.")
```

Task 3: Data Visualisation

1. Histogram of passenger ages

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
plt.figure(figsize=(10, 6))
sns.histplot(df['Age'], bins=20, kde=True)
plt.title('Distribution of Passenger Ages')
plt.xlabel('Age')
plt.ylabel('Number of Passengers')
plt.grid(axis='y', alpha=0.75)
plt.show()
```

print("Interpretation: The histogram shows that the majority of passengers are young adults, primarily in their 20s and 30s. There's a notable peak around ages 20-30, and fewer passengers in the older age groups. Children are also present in significant numbers.")

2. Bar chart showing passenger class distribution

```
plt.figure(figsize=(8, 5))
sns.countplot(x='Pclass', data=df, palette='viridis')
plt.title('Distribution of Passenger Classes')
plt.xlabel('Passenger Class (1 = 1st, 2 = 2nd, 3 = 3rd)')
plt.ylabel('Number of Passengers')
plt.xticks([0, 1, 2], ['1st Class', '2nd Class', '3rd Class'])
plt.grid(axis='y', alpha=0.75)
plt.show()
```

print("Interpretation: This bar chart illustrates the count of passengers in each class. It's clear that the majority of passengers were in 3rd class, followed by 1st class, and then 2nd class. This uneven distribution could impact survival rates.")

3. Boxplot of age by passenger class

4. Scatter plot of Age versus Fare

```
plt.figure(figsize=(10, 6))
sns.boxplot(x='Pclass', y='Age', data=df, palette='coolwarm')
plt.title('Age Distribution by Passenger Class')
plt.xlabel('Passenger Class (1 = 1st, 2 = 2nd, 3 = 3rd)')
plt.ylabel('Age')
plt.xticks([0, 1, 2], ['1st Class', '2nd Class', '3rd Class'])
plt.grid(axis='y', alpha=0.75)
plt.show()
```

print("Interpretation: The boxplot illustrates the age distribution across different passenger classes. It appears that passengers in 1st class tend to be older than those in 2nd and 3rd class. The median age generally decreases from 1st to 3rd class, suggesting that higher classes might have had older, potentially wealthier, individuals.")

```
plt.figure(figsize=(10, 6))
sns.scatterplot(x='Age', y='Fare', data=df, hue='Survived', alpha=0.7)
plt.title('Age vs. Fare with Survival Status')
plt.xlabel('Age')
plt.ylabel('Fare')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

print("Interpretation: This scatter plot visualizes the relationship between Age and Fare, with points colored by survival status. It's difficult to discern a clear linear relationship between age and fare. However, there are some high-fare outliers, many of whom survived, and a dense cluster of lower-fare passengers across various ages.")

5. Correlation Heatmap

```
plt.figure(figsize=(12, 8))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap of Numerical Features')
plt.show()
```

print("Interpretation: The heatmap displays the correlation coefficients between all numerical features. Positive values indicate a positive correlation, negative values indicate a negative correlation, and values close to zero indicate weak or no linear correlation. For example, 'Pclass' and 'Fare' show a negative correlation, suggesting that higher class tickets are generally more expensive. 'SibSp' and 'Parch' show a positive correlation, which is expected as they both relate to family size. 'Survived' shows interesting correlations with 'Pclass' (negative, higher class = more survival) and 'Fare' (positive, higher fare = more survival).")

6. Pairplot of selected numerical variables

```
numerical_vars = ['Age', 'Fare', 'SibSp', 'Parch', 'Survived']
sns.pairplot(df[numerical_vars], hue='Survived', diag_kind='kde')
plt.suptitle('Pairplot of Selected Numerical Variables by Survival Status', y=1.02)
plt.show()
```

print("Interpretation: The pairplot provides a matrix of scatter plots for each pair of numerical variables and histograms/KDEs for individual variables, colored by 'Survived' status. This allows us to observe relationships between variables and how they differ for survivors versus non-survivors. For instance, the diagonal plots show the distribution of each variable for survivors and non-survivors, highlighting differences in age, fare, and family size distributions.")

Task 4: Statistical Analysis

1. Descriptive statistics

```
print("Descriptive Statistics for Numerical Features:")
display(df.describe())
```

2. Frequency distribution

```
print("Frequency Distribution of Passenger Class:")
display(df['Pclass'].value_counts())
print("\nFrequency Distribution of Embarked Port:")
```



```
display(df['Embarked'].value_counts())
```

```
print("Interpretation: The frequency distribution for 'Pclass' confirms that 3rd class had the most passengers, followed by 1st and then 2nd class. For 'Embarked', the majority of passengers embarked from Southampton (S), followed by Cherbourg (C) and Queenstown (Q).")
```

3. Correlation analysis

```
correlation_matrix = df.corr(numeric_only=True)
print("Correlation Matrix:")
display(correlation_matrix)
```

4. Identification of the strongest positive correlation

```
strongest_positive_corr = correlation_matrix.unstack().sort_values(ascending=False)
strongest_positive_corr = strongest_positive_corr[strongest_positive_corr < 1]
strongest_positive_corr = strongest_positive_corr.drop_duplicates()
```

Filter out self-correlations and get the top one

```
positive_pairs = strongest_positive_corr[strongest_positive_corr > 0]

if not positive_pairs.empty:
    strongest_positive_pair = positive_pairs.index[0]
    strongest_positive_value = positive_pairs.iloc[0]
    print(f"Strongest positive correlation: {strongest_positive_pair} with a value of {strongest_positive_value:.2f}")
else:
    print("No positive correlations found.")
```

5. Identify the strongest negative correlation

```
strongest_negative_corr = correlation_matrix.unstack().sort_values(ascending=True)
strongest_negative_corr = strongest_negative_corr[strongest_negative_corr < 0]
strongest_negative_corr = strongest_negative_corr.drop_duplicates()
```

Filter out self-correlations and get the top one (most negative)

```
negative_pairs = strongest_negative_corr[strongest_negative_corr < 0]

if not negative_pairs.empty:
    strongest_negative_pair = negative_pairs.index[0]
    strongest_negative_value = negative_pairs.iloc[0]
    print(f"Strongest negative correlation: {strongest_negative_pair} with a value of {strongest_negative_value:.2f}")
else:
    print("No negative correlations found.")
```

6. Discuss three important statistical findings

Based on the statistical analyses performed, here are three important findings:

1. ****Passenger Class and Survival:**** There is a significant negative correlation between `Pclass` (Passenger Class) and `Survived`. This suggests that passengers in lower classes (e.g., 3rd class) had a lower chance of survival compared to those in higher classes (1st class). This is also supported by the boxplot of Age by Passenger Class, where 1st class passengers generally had higher ages and potentially more resources.
2. ****Fare and Survival:**** There is a positive correlation between `Fare` and `Survived`. This indicates that passengers who paid higher fares had a greater likelihood of survival. This finding is consistent with the `Pclass` correlation, as higher fares are typically associated with higher passenger classes, which had better survival rates.
3. ****Age Distribution:**** The distribution of `Age` shows a median around the late 20s or early 30s, with a noticeable number of children (Age < 15) and a general decrease in passenger count for older age groups. While Age itself does not have a very strong linear correlation with `Survived`, the boxplot shows that 1st class passengers tend to be older. Further analysis could explore age groups' survival rates (e.g., children vs. adults).

Task 5: Machine Learning

1. Select suitable predictor variables

For this simple classification model, we'll select a few relevant features.

'Pclass', 'Sex', 'Age', 'SibSp', 'Parch', and 'Fare' are good candidates.

'Embarked' is also categorical and potentially useful.

We need to convert categorical features into numerical format for the model.

Convert 'Sex' and 'Embarked' into numerical format using one-hot encoding

```
df_ml = df.copy()
df_ml = pd.get_dummies(df_ml, columns=['Sex', 'Embarked'], drop_first=True)
```

Select predictor variables (features) and the target variable

```
X = df_ml[['Pclass', 'Age', 'SibSp', 'Parch', 'Fare', 'Sex_male', 'Embarked_Q', 'Embarked_S']]
y = df_ml['Survived']
```

```
print("Selected predictor variables (features):")
display(X.head())
print("\nTarget variable (Survived):")
display(y.head())
```

2. Split the dataset into training and testing sets

```
from sklearn.model_selection import train_test_split
```

Split the data into training and testing sets (80% train, 20% test)

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
print(f"Training features shape: {X_train.shape}")
print(f"Testing features shape: {X_test.shape}")
print(f"Training target shape: {y_train.shape}")
```

```
print(f"Testing target shape: {y_test.shape}")
```

```
from sklearn.impute import SimpleImputer
```

Initialize the SimpleImputer with the median strategy

The median is chosen to be robust to outliers in the 'Age' column.

```
imputer = SimpleImputer(strategy='median')
```

Fit the imputer on X_train and transform both X_train and X_test

```
X_train_imputed = imputer.fit_transform(X_train)
```

```
X_test_imputed = imputer.transform(X_test)
```

Convert the imputed arrays back to DataFrames, preserving column names

```
X_train = pd.DataFrame(X_train_imputed, columns=X_train.columns)
```

```
X_test = pd.DataFrame(X_test_imputed, columns=X_test.columns)
```

```
print("Missing values in X_train after imputation:")
```

```
print(X_train.isnull().sum())
```

```
print("\nMissing values in X_test after imputation:")
```

```
print(X_test.isnull().sum())
```

```
print("X_train and X_test imputed successfully.")
```

3. Train a Logistic Regression classifier

```
from sklearn.linear_model import LogisticRegression
```

Initialize the Logistic Regression model

Set max_iter to a higher value if it fails to converge

```
model = LogisticRegression(random_state=42, solver='liblinear', max_iter=200)
```

Train the model on the training data

```
model.fit(X_train, y_train)
```

```
print("Logistic Regression model trained successfully.")
```

4. Predict the testing data

Predict on the test set

```
y_pred = model.predict(X_test)
```

```
print("Predictions on the testing data generated successfully.")
```

5. Compute Accuracy, Confusion Matrix, Classification Report

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Compute Accuracy

```
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy:.2f}")
```

Compute Confusion Matrix

```
conf_matrix = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
display(pd.DataFrame(conf_matrix, index=['Actual No Survival', 'Actual Survival'],
columns=['Predicted No Survival', 'Predicted Survival']))
```

Compute Classification Report

```
class_report = classification_report(y_test, y_pred)
print("\nClassification Report:")
print(class_report)
```

6. Discuss model performance

The Logistic Regression model achieved an accuracy of approximately XX%. Looking at the Confusion Matrix:

```
* **True Positives (TP):** The number of actual survivors correctly predicted as survivors.
* **True Negatives (TN):** The number of actual non-survivors correctly predicted as non-survivors.
* **False Positives (FP):** The number of actual non-survivors incorrectly predicted as survivors (Type I error).
* **False Negatives (FN):** The number of actual survivors incorrectly predicted as non-survivors (Type II error).
```

The Classification Report provides further insights:

```
* **Precision:** For each class, it indicates the proportion of correctly predicted positive observations out of all positive predictions. A high precision means fewer false positives.
* **Recall (Sensitivity):** For each class, it indicates the proportion of correctly predicted positive observations out of all actual positives. A high recall means fewer false negatives.
* **F1-Score:** The harmonic mean of precision and recall. It's a good measure to evaluate the model when there's an uneven class distribution.
* **Support:** The number of actual occurrences of the class in the specified dataset.
```

Overall, the model's performance can be evaluated by these metrics. For instance, if predicting survival is critical, we might prioritize recall for the 'survived' class to minimize false negatives (i.e., people who survived but were predicted not to). The current model provides a baseline performance, which could be improved by feature engineering, trying other models, or hyperparameter tuning.

Task 6: Discussion and Conclusion

This section should synthesize all the work performed throughout the project. It should be approximately two pages in length and cover the following aspects:

```

### Major Findings
* Summarize the key insights gained from the data acquisition, cleaning, and exploratory
data analysis phases.
* Highlight interesting patterns, distributions, or relationships discovered in the
visualizations.

### Statistical Insights
* Discuss the meaning and implications of the descriptive statistics.
* Elaborate on the significant correlations identified and their potential impact on
passenger survival or other variables.
* Explain any notable frequency distributions and what they tell you about the dataset.

### Machine Learning Results
* Interpret the performance metrics of your Logistic Regression model (Accuracy, Confusion
Matrix, Classification Report).
* Discuss what the precision, recall, and F1-score tell you about the model's ability to
predict survival for both classes.
* Consider if the model's performance is acceptable for the problem and what factors might
be contributing to its strengths or weaknesses.

### Limitations of the Study
* Identify any constraints or shortcomings in your data, methods, or model.
* Discuss potential biases in the dataset or limitations of the features used.
* Consider aspects that could not be fully explored due to data availability or project
scope.

### Recommendations
* Suggest future work or improvements based on your findings and limitations.
* This could include ideas for further feature engineering, trying more advanced machine
learning models, collecting additional data, or conducting more in-depth statistical tests.
strongest_negative_corr = correlation_matrix.unstack().sort_values(ascending=True)
strongest_negative_corr = strongest_negative_corr[strongest_negative_corr < 0]
strongest_negative_corr = strongest_negative_corr.drop_duplicates()

Filter out self-correlations and get the top one (most negative)
negative_pairs = strongest_negative_corr[strongest_negative_corr < 0]

if not negative_pairs.empty:
    strongest_negative_pair = negative_pairs.index[0]
    strongest_negative_value = negative_pairs.iloc[0]
    print(f"Strongest negative correlation: {strongest_negative_pair} with a value of
{strongest_negative_value:.2f}")
else:
    print("No negative correlations found.")

plt.figure(figsize=(10, 6))
sns.boxplot(x='Pclass', y='Age', data=df, palette='coolwarm')
plt.title('Age Distribution by Passenger Class')
plt.xlabel('Passenger Class (1 = 1st, 2 = 2nd, 3 = 3rd)')
plt.ylabel('Age')
plt.xticks([0, 1, 2], ['1st Class', '2nd Class', '3rd Class'])
plt.grid(axis='y', alpha=0.75)
plt.show()

print("Interpretation: The boxplot illustrates the age distribution across different passenger
classes. It appears that passengers in 1st class tend to be older than those in 2nd and 3rd
class. The median age generally decreases from 1st to 3rd class, suggesting that higher
classes might have had older, potentially wealthier, individuals.")

```